TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

KHOA TOÁN - TIN HỌC



BÁO CÁO THỰC HÀNH

Môn học: Xử lý dữ liệu lớn

Bài thực hành L04

Nạp dữ liệu từ cơ sở dữ liệu OLTP vào HDFS: Sqoop và Spark JDBC, nạp toàn phần và nạp gia tăng

Sinh viên thực hiện: Nguyễn Thị Huyền Trang
Mã số sinh viên: 23280092
Lớp: 23KDL

TP. Hồ Chí Minh, Tháng 10 năm 2026

Mục lục

1 Mục tiêu và môi trường **2**

2 Thực hiện và kết quả **4**

2.1 Sqoop nạp toàn phần bảng `orders` 4
2.2 Sqoop với cột chia bị lệch 6
2.3 Sqoop nạp gia tăng, saved job, merge và export 7
2.4 Spark JDBC: đọc song song và ghi Parquet 10
2.5 Spark nạp gia tăng bằng watermark 13
2.6 So sánh Sqoop và Spark JDBC 16

3 Trả lời câu hỏi checkpoint **16**

3.1 Sqoop chạy câu SQL nào trước khi import, và từ đó ra 4 split bằng cách nào? . 16
3.2 Vì sao các file `customers` không đều, và nếu dùng `--split-by customer_id` thì sao? 17
3.3 Với `lowerBound=1, upperBound=5003, numPartitions=4`, Spark có đọc dòng `order_seq` = 9,999 không? 17
3.4 Vì sao phải chốt cận trên trước khi đọc, và chỉ lưu watermark sau khi ghi xong? 17
3.5 Chế độ `append` hay `lastmodified` bắt được UPDATE, và chế độ nào bắt được DELETE? 18
3.6 Watermark trên `updated_at` không giải quyết được gì, và CDC khác ở đâu? . . 18
3.7 Điều gì xảy ra với cơ sở dữ liệu nguồn khi đặt `-m 50` hoặc `numPartitions=50`? 18

4 Kết luận **18**

Tài liệu tham khảo **19**

Phụ lục: Tệp nộp kèm **19**

Danh mục hình

1 Trạng thái container của stack L04 và 6 bảng nguồn trong PostgreSQL. 3
2 Sqoop nạp toàn phần `orders`: bounding query, 4 split và số dòng mỗi file. . . . 5
3 Nhật ký PostgreSQL ghi lại bounding query và 4 câu SELECT theo khoảng `order_seq` do các map task của Sqoop gửi. 5
4 JobHistory của job `job_1793356967101_0001` (`orders.jar`): 4 map task thành công, không có reduce task. 6
5 Số dòng mỗi file khi nạp `customers` chia theo `customer_zip_code_prefix`, thể hiện dữ liệu bị lệch. 7
6 Sqoop nạp `append` lần đầu, nạp `lastmodified` cơ sở và tạo saved job `orders_incr`. 8

7 Sau `simulate_changes` lần 1: saved job, `lastmodified` kèm merge và export về PostgreSQL. . . . 9
8 YARN ResourceManager UI: các MapReduce job do Sqoop nộp, tất cả ở trạng thái FINISHED/SUCCEEDED. . . . 9
9 JobHistory Server liệt kê các job Sqoop đã hoàn tất. . . . 10
10 Nhật ký PostgreSQL ghi lại các câu `SELECT 1 FROM orders WHERE ...` theo khoảng do Spark sinh ra. . . . 11
11 Kết quả `spark-submit --step full`, gồm physical plan có `PartitionFilters` . . . 12
12 Spark UI, trang Jobs của ứng dụng `L04-jdbc-full`. . . . 12
13 Spark UI, trang Stages: các stage đọc JDBC song song và stage ghi Parquet đều có 4 task. . . . 13
14 Đầu ra Parquet phân vùng theo tháng trên HDFS và so sánh dung lượng với đầu ra văn bản của Sqoop. . . . 13
15 Ba lần chạy Spark `--step incremental`. . . . 15

Danh mục bảng

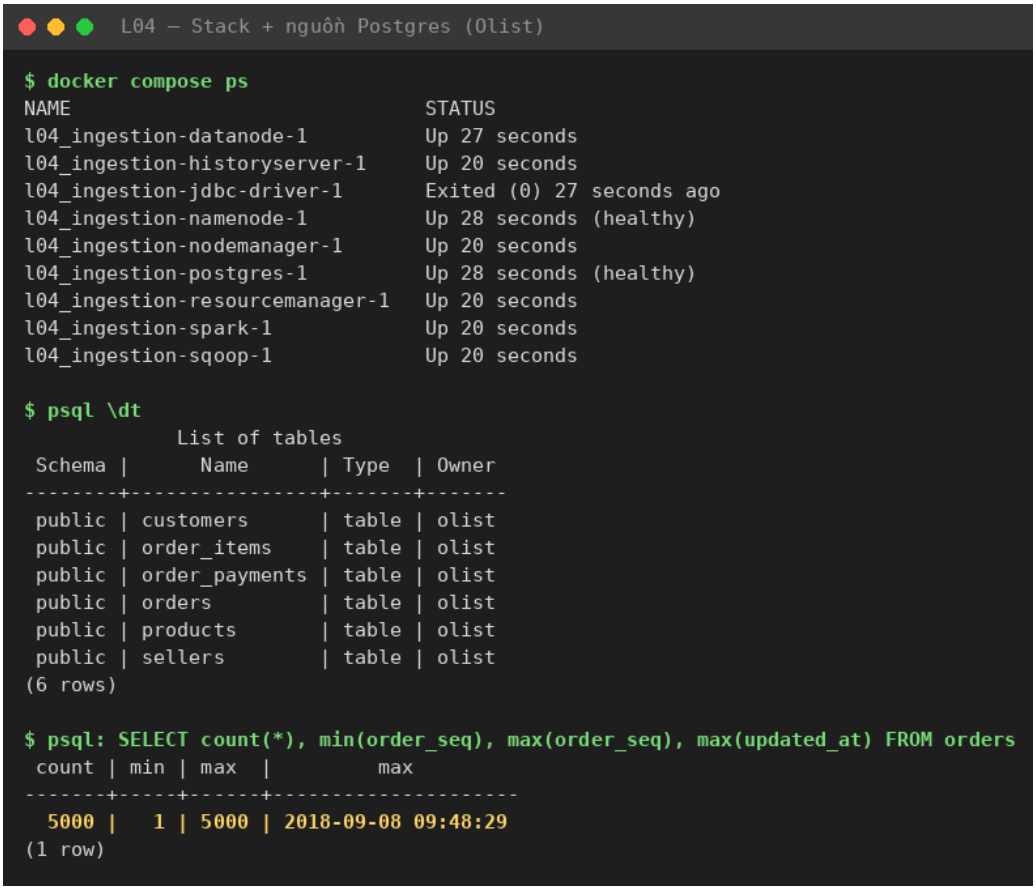
1 Môi trường thực nghiệm (stack `l04_ingestion`). . . . 3
2 Phân bố trạng thái của 5,000 đơn hàng ban đầu. . . . 4
3 Các split của Sqoop khi nạp `orders` với `--split-by order_seq -m 4`. . . . 4
4 Số dòng mỗi file khi chia theo cột phân bố đều và cột bị lệch. . . . 6
5 Phân bố `customer_zip_code_prefix` theo `width_bucket(..., 1003, 99990, 4)`. . . . 7
6 Kết quả các bước nạp gia tăng, merge và export của Sqoop. . . . 7
7 So sánh các cách đọc bảng `orders` bằng Spark JDBC. . . . 10
8 Số dòng các bảng phụ được Spark nạp sang Parquet. . . . 11
9 Kết quả ba lần chạy Spark nạp gia tăng (thời điểm ngày 2026-10-30). . . . 14
10 So sánh Sqoop và Spark JDBC khi nạp bảng `orders`. . . . 16
11 Danh mục tệp nộp kèm báo cáo L04. . . . 19

1 Mục tiêu và môi trường

Bài thực hành nạp dữ liệu từ PostgreSQL vào HDFS bằng Apache Sqoop 1.4.7 (đã vào Apache Attic từ tháng 6 năm 2021) [1] và Spark JDBC [2], theo hai cách nạp toàn phần và nạp gia tăng, rồi so sánh hai công cụ. PostgreSQL chạy với `log_statement=all` để đọc trực tiếp câu SQL mà mỗi công cụ gửi xuống. Hệ thống chạy bằng Docker Compose trên một máy (Bảng 1). Container `jdbc-driver` chỉ tải driver rồi thoát với mã 0; ban đầu chúng tôi tưởng nó lỗi, nhưng đó là hành vi đúng (Hình 1).

Bảng 1. Môi trường thực nghiệm (stack `l04_ingestion`).

Thành phần	Phiên bản / cấu hình
PostgreSQL	18.6, `log_statement=all`, cơ sở dữ liệu nguồn Olist
HDFS	3.4.3, 1 NameNode + 1 DataNode
YARN	ResourceManager, NodeManager, JobHistory Server (chạy job của Sqoop)
Sqoop	1.4.7 (image khóa học `de-labs/sqoop:1.4.7`)
Spark	4.1.3, local mode, Spark UI cổng 4040
JDBC driver	PostgreSQL JDBC (`/jars/postgresql.jar`), dùng chung cho Sqoop và Spark



Hình 1. Trạng thái container của stack L04 và 6 bảng nguồn trong PostgreSQL.

Nguồn là cơ sở dữ liệu Olist tổng hợp gồm 6 bảng (`customers`, `orders`, `order_items`, `order_payments`, `products`, `sellers`). Bảng `orders` có 5,000 dòng, `order_seq` chạy từ 1 đến 5000, `max(updated_at)` là `2018-09-08 09:48:29`. `order_seq` là khóa thay thế kiểu số, tăng theo thời gian mua, còn `order_id`, `customer_id` là chuỗi hash 32 ký tự. Mỗi bảng có cột

`updated_at` do trigger tự cập nhật khi UPDATE. Phân bố trạng thái đơn hàng lấy bằng `sqoop eval` nằm ở Bảng 2.

Bảng 2. Phân bố trạng thái của 5,000 đơn hàng ban đầu.

`order_status`	Số đơn
delivered	4,793
shipped	102
unavailable	41
canceled	30
processing	19
invoiced	11
approved	3
created	1

Script `sql/simulate_changes.sql` chèn 3 khách hàng và 3 đơn mới, đổi 2 đơn `shipped` đầu tiên thành `delivered`. Chúng tôi chạy nó hai lần: trước phần Sqoop gia tăng và giữa hai lần chạy Spark gia tăng. Vì thế lúc chạy Spark, `orders` và `customers` đã có 5,003 dòng, khác giá trị tham chiếu trong README.

2 Thực hiện và kết quả

2.1 Sqoop nạp toàn phần bảng `orders`

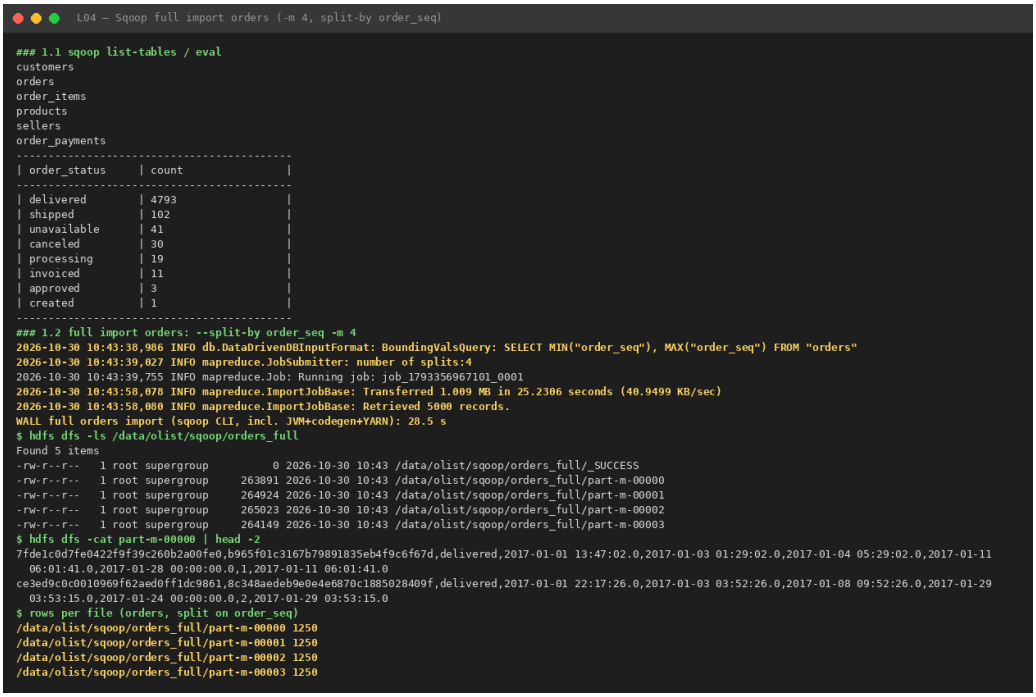
Mỗi lệnh `import` được biên dịch thành một MapReduce job chỉ có pha map [1]. Với `--split-by order_seq -m 4`, Sqoop báo `number of splits:4` và nhật ký PostgreSQL có đúng 4 câu SELECT theo khoảng của 4 map (Bảng 3, Hình 3; cách chia giải thích ở Mục 3.1). Job `job_1793356967101_0001` báo `Transferred 1.009 MB in 25.2306 seconds (40.9499 KB/sec)` và `Retrieved 5000 records`; cả lệnh CLI mất 28.5 s vì còn khởi động JVM, sinh và biên dịch lớp record rồi nộp job lên YARN.

Bảng 3. Các split của Sqoop khi nạp `orders` với `--split-by order_seq -m 4`.

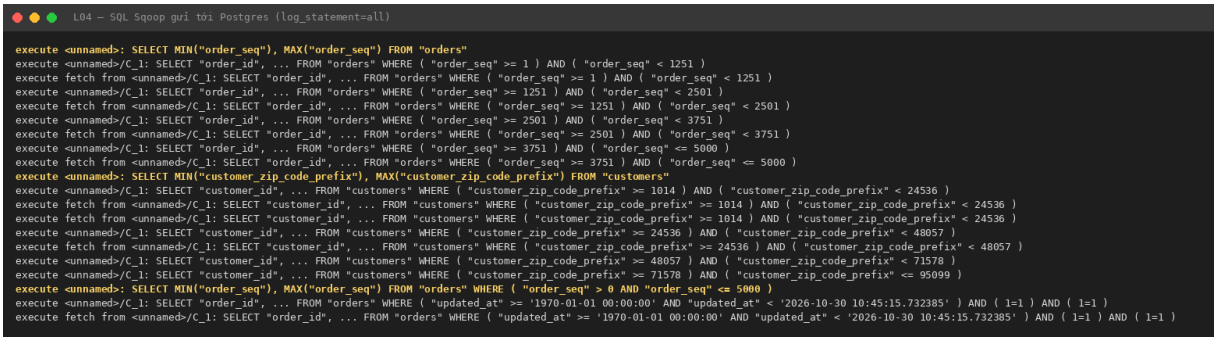
Map	Điều kiện WHERE	Số dòng	Kích thước file
0	`order_seq >= 1 AND order_seq < 1251`	1,250	263,891 B
1	`order_seq >= 1251 AND order_seq < 2501`	1,250	264,924 B
2	`order_seq >= 2501 AND order_seq < 3751`	1,250	265,023 B
3	`order_seq >= 3751 AND order_seq <= 5000`	1,250	264,149 B

Bốn file `part-m-*` gần bằng nhau cả về số dòng lẫn kích thước (Hình 2), dạng văn bản CSV với

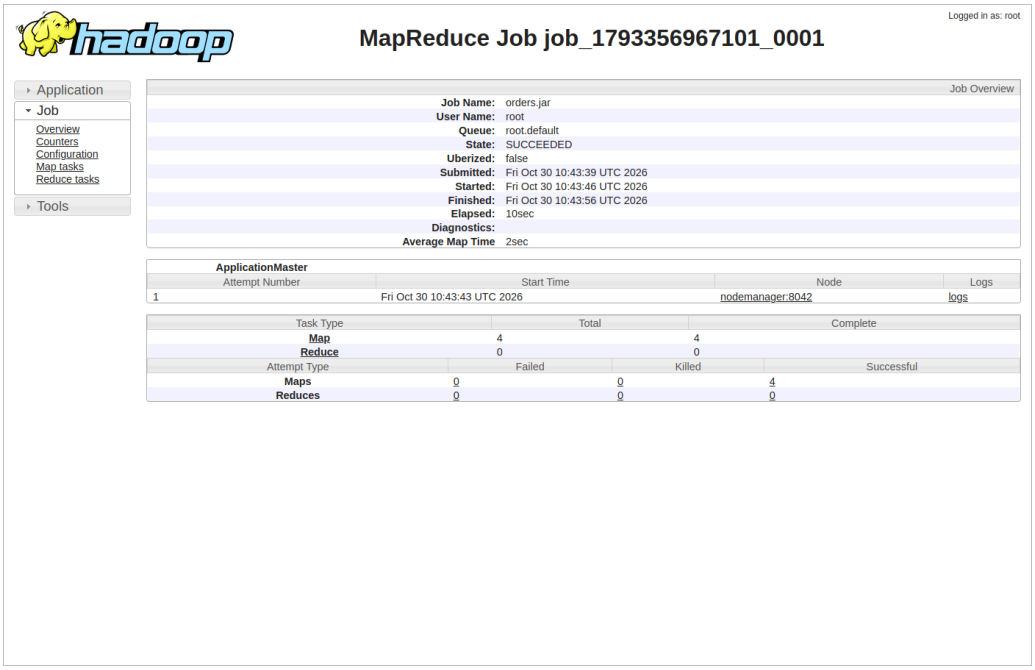
timestamp `2017-01-03 01:29:02.0` và NULL ghi là `\N`. JobHistory xác nhận 4 map, 0 reduce (Hình 4).



Hình 2. Sqoop nạp toàn phần `orders`: bounding query, 4 split và số dòng mỗi file.



Hình 3. Nhật ký PostgreSQL ghi lại bounding query và 4 câu SELECT theo khoảng `order_seq` do các map task của Sqoop gửi.



Hình 4. JobHistory của job `job_1793356967101_0001` (`orders.jar`): 4 map task thành công, không có reduce task.

2.2 Sqoop với cột chia bị lệch

Bảng `customers` được nạp với `--split-by customer_zip_code_prefix -m 4` vào `/data/olist/sqoop/customers_full`. Bounding query trả về (1014, 95099), và nhật ký PostgreSQL cho thấy 4 khoảng rộng xấp xỉ 23,521 với các điểm cắt 24536, 48057 và 71578. Số dòng mỗi file đặt cạnh trường hợp `orders` ở Bảng 4.

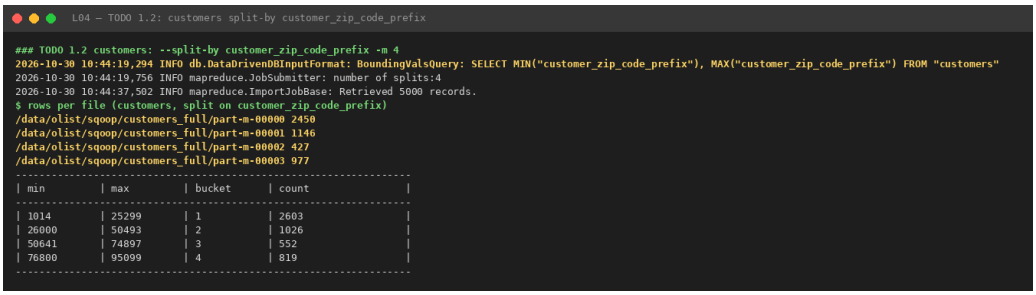
Bảng 4. Số dòng mỗi file khi chia theo cột phân bố đều và cột bị lệch.

Bảng / cột split	part-m-00000	part-m-00001	part-m-00002	part-m-00003
`orders` / `order_seq` (1–5000)	1,250	1,250	1,250	1,250
`customers` / `customer_zip_code_prefix` (1014–95099)	**2,450**	1,146	**427**	977

Split đầu nhận gần nửa bảng, gấp khoảng 5.7 lần split thứ ba (Hình 5). Đếm độc lập bằng `width_bucket` trên 4 thùng (Bảng 5) cho cùng xu hướng: thùng chứa vùng São Paulo (01xxx–19xxx) chiếm phần lớn khách hàng.

Bảng 5. Phân bố `customer_zip_code_prefix` theo `width_bucket(..., 1003, 99990, 4)`.

Thùng	min zip	max zip	Số khách hàng
1	1014	25299	2,603
2	26000	50493	1,026
3	50641	74897	552
4	76800	95099	819



Hình 5. Số dòng mỗi file khi nạp `customers` chia theo `customer_zip_code_prefix`, thể hiện dữ liệu bị lệch.

2.3 Sqoop nạp gia tăng, saved job, merge và export

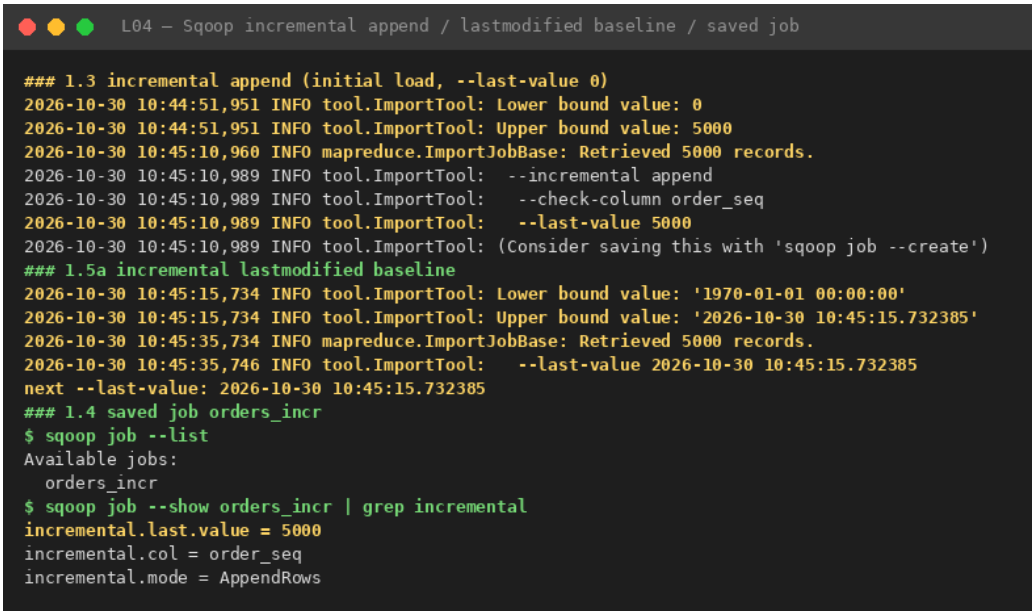
Chuỗi thao tác được tóm ở Bảng 6. Lần `append` đầu (`--last-value 0`) có điều kiện `order_seq > 0 AND order_seq <= 5000`, hai split `[1, 2501)` và `[2501, 5000]`. Saved job `orders_incr` lưu `incremental.col = order_seq` và `incremental.mode = AppendRows` trong metastore của Sqoop.

Bảng 6. Kết quả các bước nạp gia tăng, merge và export của Sqoop.

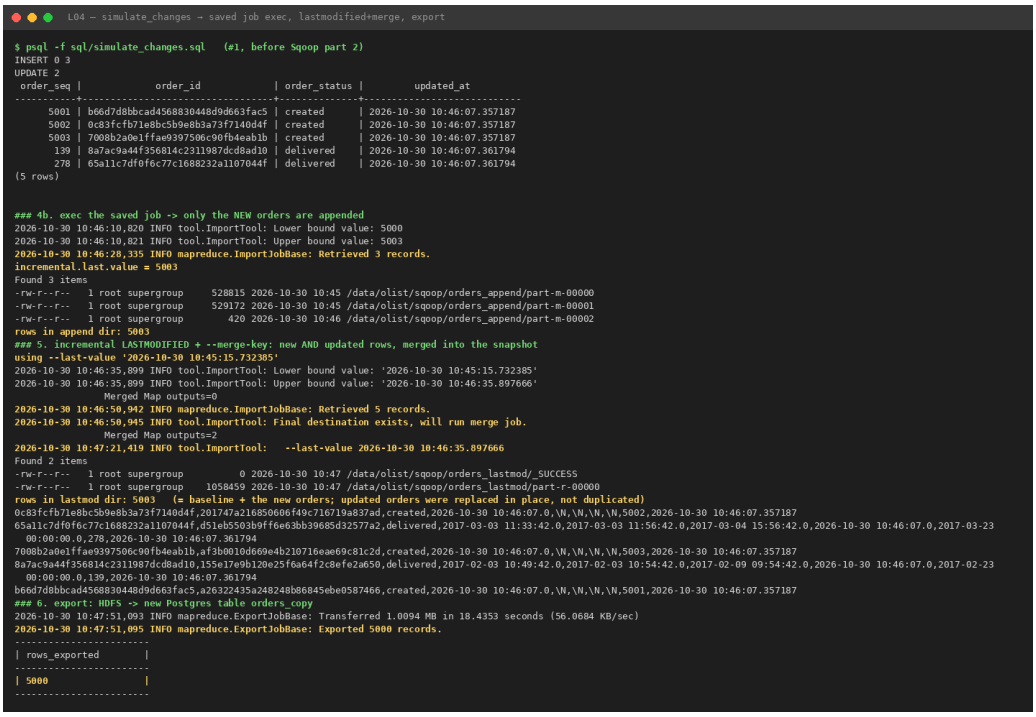
Bước	Cửa sổ	Kết quả
`append`, lần đầu (`-m 2`)	lower 0, upper 5000	5,000 dòng, gợi ý `--last-value 5000`
`lastmodified`, cơ sở (`-m 1`)	`'1970-01-01 00:00:00'` đến `'2026-10-30 10:45:15.732385'`	5,000 dòng
Saved job `orders_incr`		`incremental.last.value = 5000`
`simulate_changes` lần 1		tạo `order_seq` 5001–5003; đơn 139 và 278 chuyển `delivered`
`sqoop job --exec orders_incr`	lower 5000, upper 5003	**3** dòng; `last.value` thành 5003; thư mục có 5,003 dòng

Bước	Cửa sổ	Kết quả
`lastmodified` + `--merge-key order_id` `sqoop export` sang `orders_copy` (`-m 2`)	`'...10:45:15.732385'` đến `'...10:46:35.897666'`	**5** dòng; merge job tạo `part-r-00000` 5,003 dòng 1.0094 MB trong 18.4353 s, 5,000 record

Saved job chạy `... WHERE ( "order_seq" > 5000 AND "order_seq" <= 5003 )` và chỉ thêm một file `part-m-00002` (420 B) cạnh hai file 528,815 B và 529,172 B của lần đầu; đơn 139 và 278 vừa cập nhật không được nạp lại. Với `lastmodified`, do thư mục đích đã tồn tại, Sqoop chạy thêm một merge job (`Merged Map outputs=2`) thay bản cũ của cùng `order_id` bằng bản mới. Kết quả là một file `part-r-00000` (1,058,459 B) không trùng lặp, trong đó đơn 139 và 278 mang trạng thái `delivered` và `updated_at` mới. Một chi tiết chúng tôi chỉ thấy khi đọc log: `lastmodified` dùng cửa sổ nửa mở `updated_at >= lower AND updated_at < upper`, ngược với Spark ở phần sau (Hình 6, Hình 7).

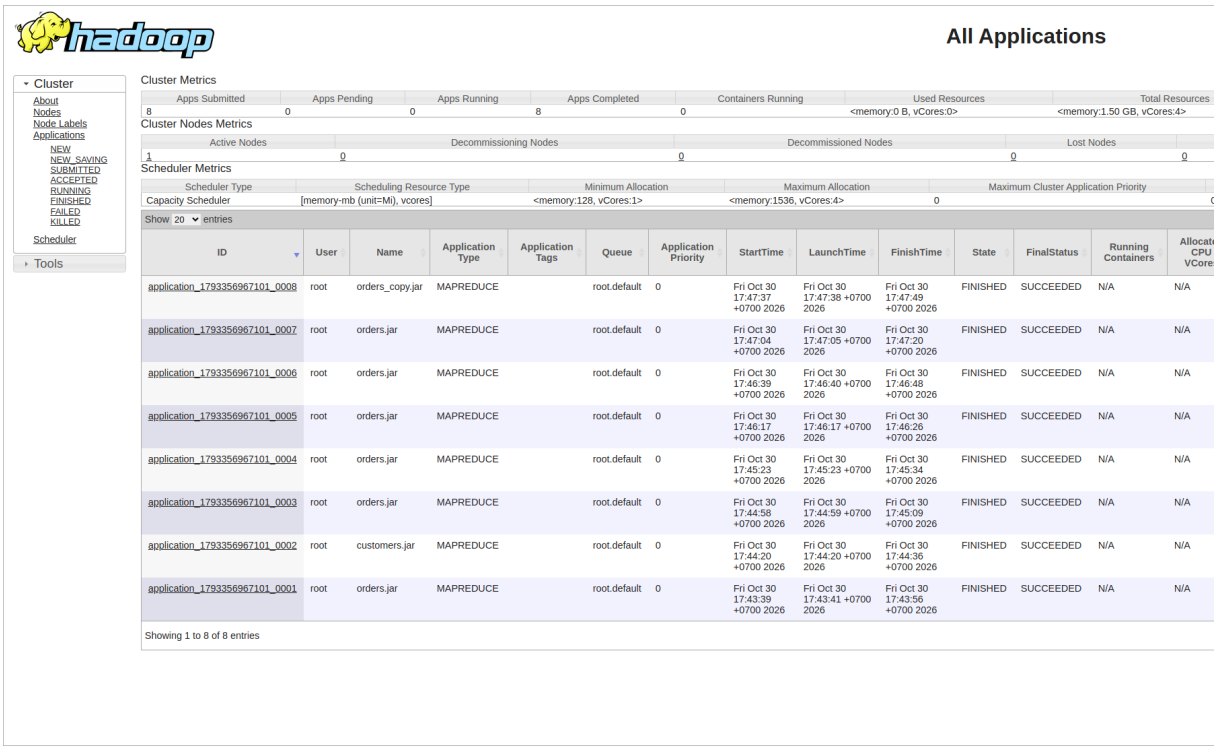


Hình 6. Sqoop nạp append lần đầu, nạp `lastmodified` cơ sở và tạo saved job `orders_incr`.

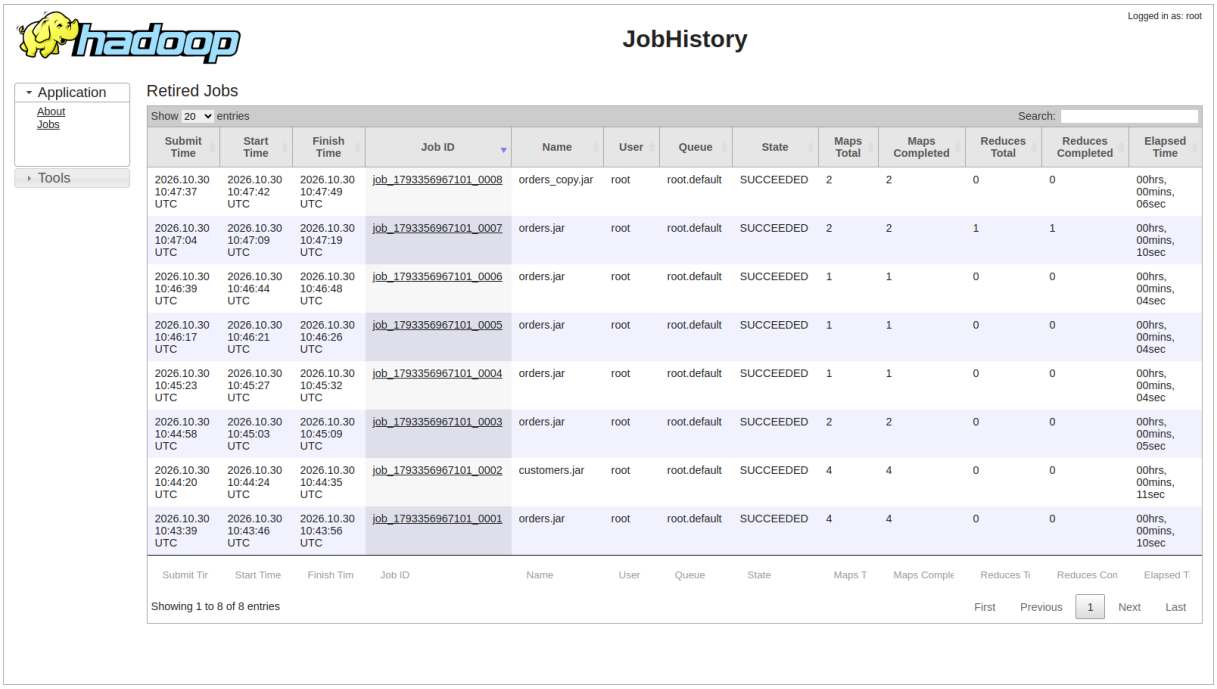


Hình 7. Sau `simulate_changes` lần 1: saved job, `lastmodified` kèm merge và export về PostgreSQL.

ResourceManager liệt kê 8 ứng dụng MapReduce do Sqoop nộp (`orders.jar`, `customers.jar`, `orders_copy.jar`), đều thành công (Hình 8, Hình 9).



Hình 8. YARN ResourceManager UI: các MapReduce job do Sqoop nộp, tất cả ở trạng thái FINISHED/SUCCEEDED.



Hình 9. JobHistory Server liệt kê các job Sqoop đã hoàn tất.

2.4 Spark JDBC: đọc song song và ghi Parquet

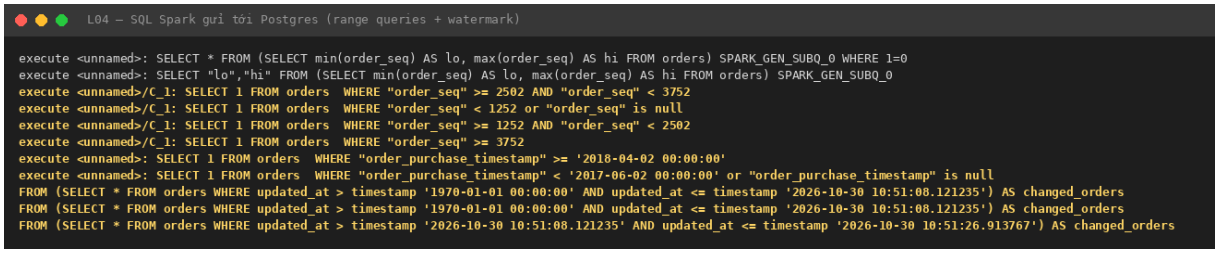
Nguồn JDBC của Spark nhận `partitionColumn`, `lowerBound`, `upperBound` và `numPartitions`, tính bước $s = (\text{upper} - \text{lower})/n$ rồi sinh n mệnh đề WHERE; partition đầu và cuối không có cận ngoài [2]. Script lấy cận thật bằng `SELECT min(order_seq) AS lo, max(order_seq) AS hi FROM orders`, nhận `1 .. 5003`, rồi đọc theo ba cách (Bảng 7).

Bảng 7. So sánh các cách đọc bảng `orders` bằng Spark JDBC.

Cách đọc	Số partition	Thời gian `count()`	Số dòng mỗi partition
Naive (không có `partitionColumn`)	1	1.17 s	5,003 dòng trong 1 task
`partitionColumn=order_seq`, bounds 1..5003	4	0.22 s	1,251 / 1,250 / 1,250 / 1,252
`partitionColumn=order_purchase_timestamp`, 2017-01-01 đến 2018-09-01	4	không đo	782 / 1,116 / 1,419 / 1,686

Nhật ký PostgreSQL (Hình 10) có 4 câu do Spark sinh: `WHERE "order_seq" < 1252 or "order_seq" is null`, `>= 1252 AND < 2502`, `>= 2502 AND < 3752` và `WHERE "order_`

`seq" >= 3752`. Bước chia là $(5003-1)/4 \approx 1250$, nên partition đầu có 1,251 dòng và partition cuối 1,252 dòng. Vì lệnh là `count()`, Spark chỉ gửi `SELECT 1` (column pruning đẩy xuống SQL). Với cột timestamp, partition đầu là `< '2017-06-02 00:00:00' or ... is null`, partition cuối là `>= '2018-04-02 00:00:00'`, và số dòng tăng dần vì lượng đơn Olist tăng theo thời gian. Thời gian `count()` chỉ đo một lần, lần song song chạy sau lần naive nên có thể được lợi từ JVM đã nóng; con số 0.22 s chưa đủ để kết luận về hiệu năng.

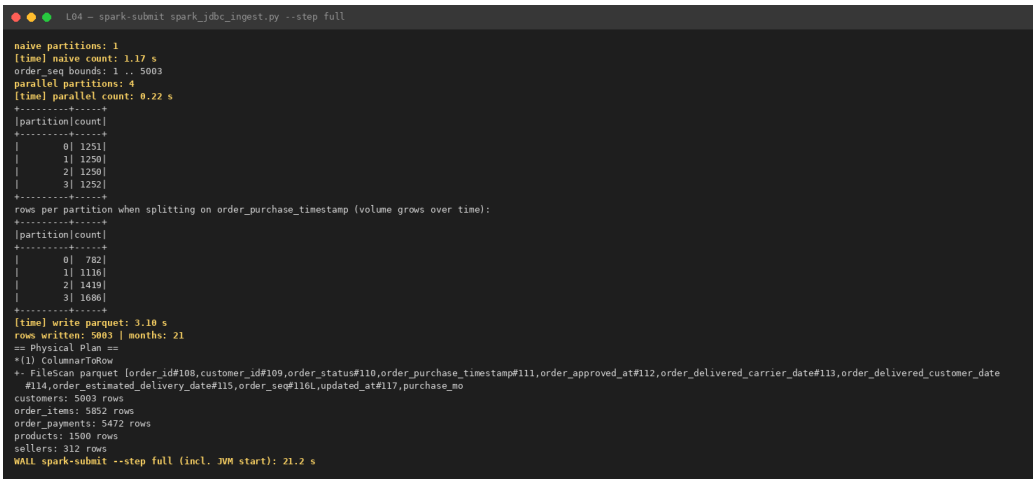


Hình 10. Nhật ký PostgreSQL ghi lại các câu `SELECT 1 FROM orders WHERE ...` theo khoảng do Spark sinh ra.

Ghi Parquet với `partitionBy("purchase_month")` mất 3.10 s, tạo 21 thư mục tháng (20 tháng lịch sử và tháng của các đơn mới) chứa đủ 5,003 dòng. Truy vấn lọc `purchase_month = '2018-08'` có physical plan chứa `PartitionFilters: [isnotnull(purchase_month#118), (purchase_month#118 = 2018-08)]`, tức partition pruning có tác dụng. Năm bảng còn lại được nạp mỗi bảng một partition (Bảng 8). Cả `spark-submit --step full` mất 21.2 s (Hình 11).

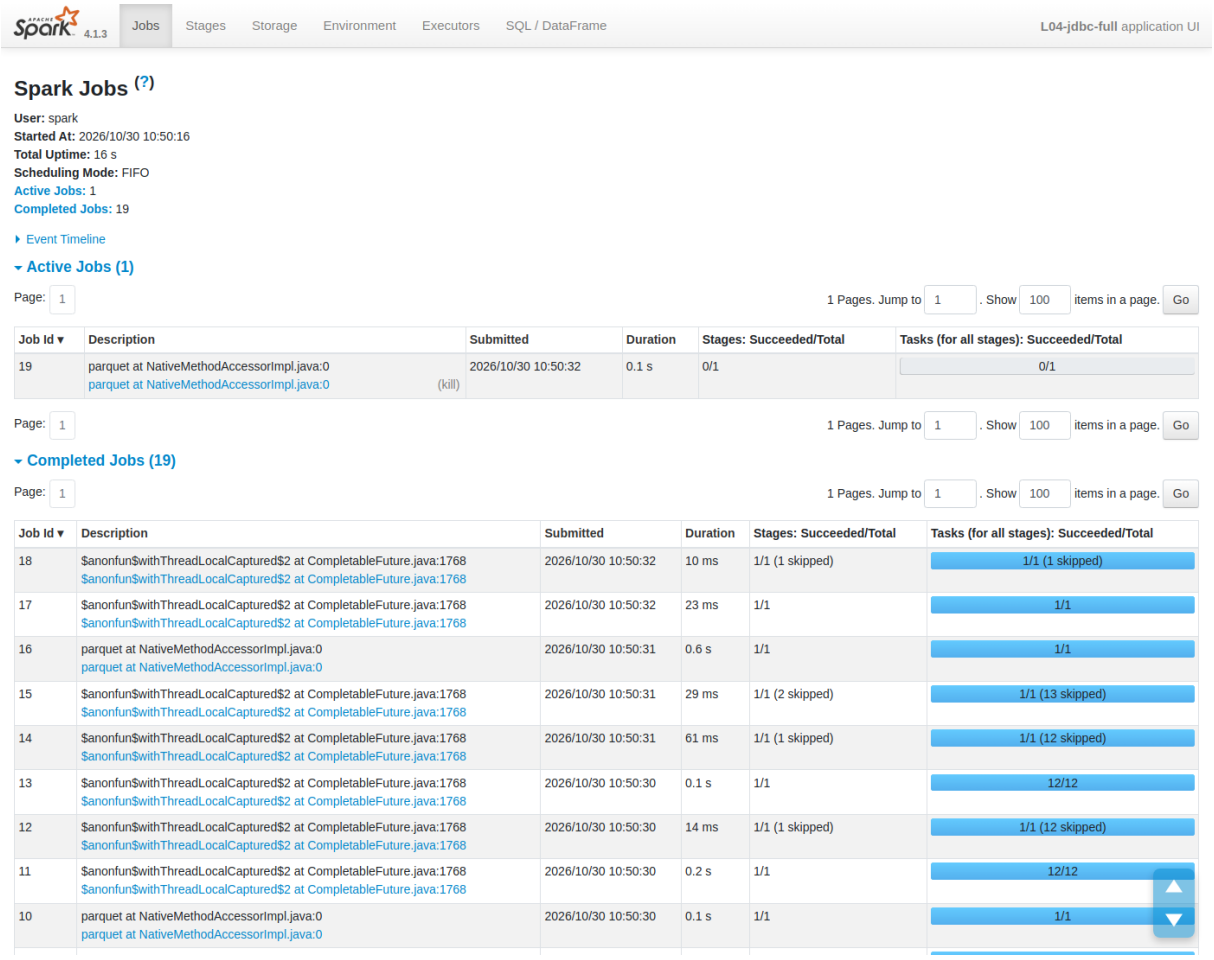
Bảng 8. Số dòng các bảng phụ được Spark nạp sang Parquet.

Bảng	Số dòng
customers	5,003
order_items	5,852
order_payments	5,472
products	1,500
sellers	312

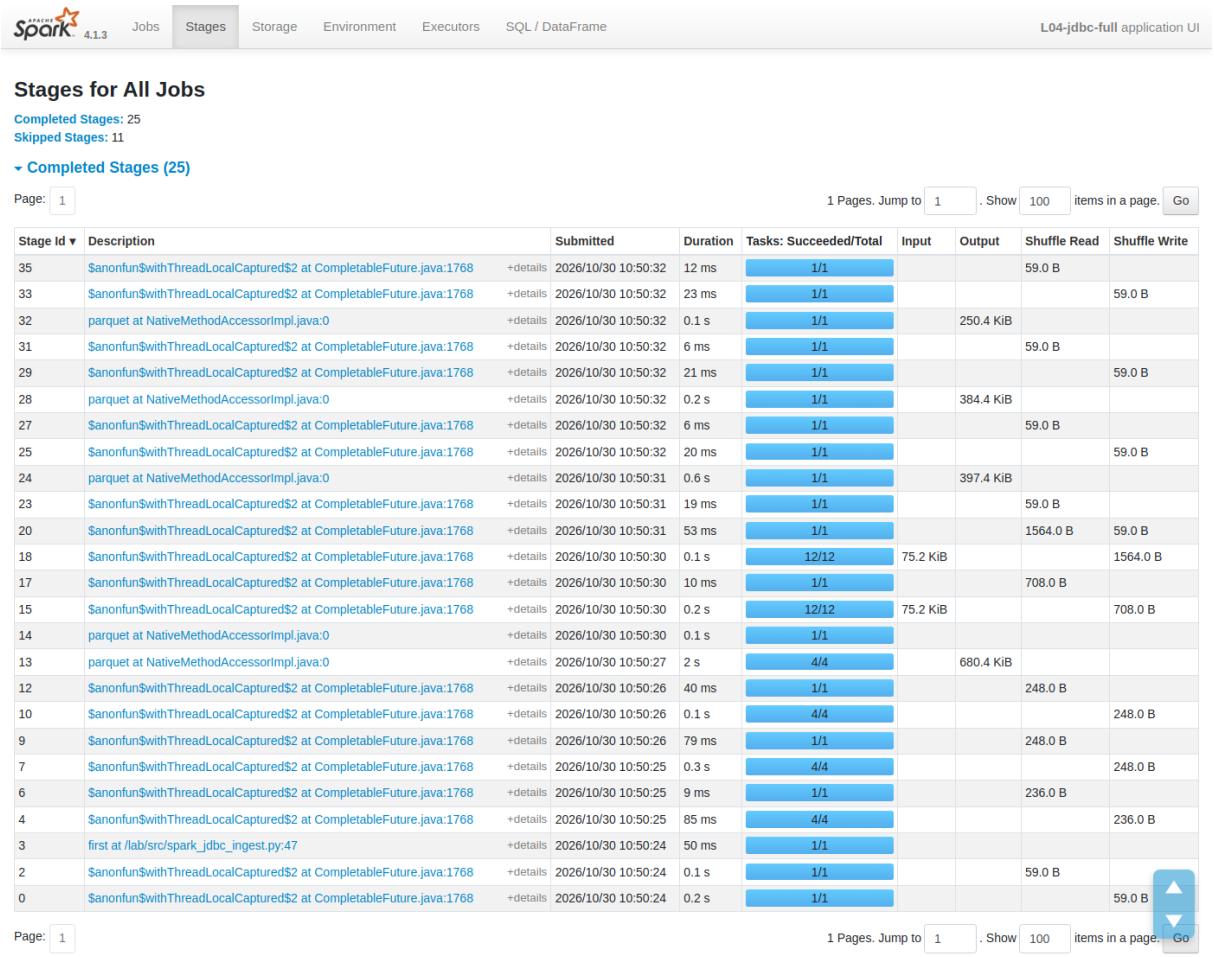


Hình 11. Kết quả `spark-submit --step full`, gồm physical plan có `PartitionFilters`.

Trên Spark UI, các stage đọc song song và stage ghi Parquet đều có 4/4 task, đầu ra 680.4 KiB (Hình 12, Hình 13). Trên HDFS, `/data/olist/spark/orders` có 22 mục (`_SUCCESS` và 21 thư mục `purchase_month=`), dung lượng 680.4 K, còn `/data/olist/sqoop/orders_full` là 1.0 M (Hình 14).

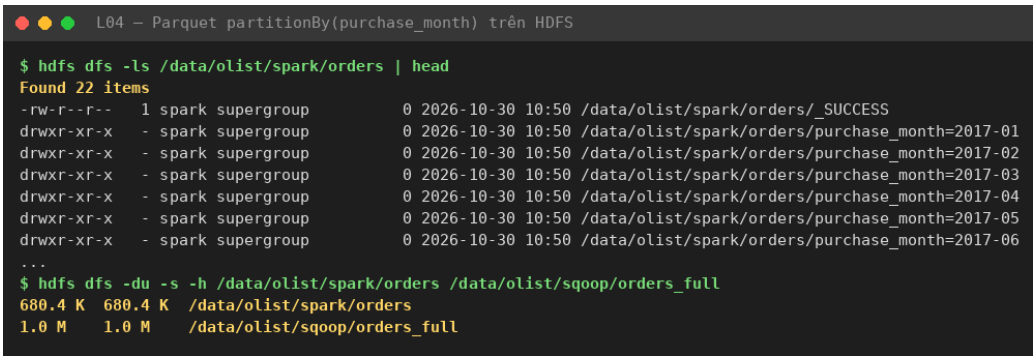


Hình 12. Spark UI, trang Jobs của ứng dụng `L04-jdbc-full`.



Stage Id ▾	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
35	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	12 ms	1/1			59.0 B	
33	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	23 ms	1/1				59.0 B
32	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:50:32	0.1 s	1/1		250.4 KiB		
31	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	6 ms	1/1			59.0 B	
29	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	21 ms	1/1				59.0 B
28	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:50:32	0.2 s	1/1		384.4 KiB		
27	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	6 ms	1/1			59.0 B	
25	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:32	20 ms	1/1				59.0 B
24	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:50:31	0.6 s	1/1		397.4 KiB		
23	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:31	19 ms	1/1			59.0 B	
20	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:31	53 ms	1/1			1564.0 B	59.0 B
18	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:30	0.1 s	12/12	75.2 KiB			1564.0 B
17	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:30	10 ms	1/1			708.0 B	
15	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:30	0.2 s	12/12	75.2 KiB			708.0 B
14	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:50:30	0.1 s	1/1				
13	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/30 10:50:27	2 s	4/4		680.4 KiB		
12	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:26	40 ms	1/1			248.0 B	
10	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:26	0.1 s	4/4				248.0 B
9	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:26	79 ms	1/1			248.0 B	
7	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:25	0.3 s	4/4				248.0 B
6	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:25	9 ms	1/1			236.0 B	
4	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:25	85 ms	4/4				236.0 B
3	first at /lab/src/spark_jdbc_ingest.py:47 +details	2026/10/30 10:50:24	50 ms	1/1				
2	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:24	0.1 s	1/1			59.0 B	
0	$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768 +details	2026/10/30 10:50:24	0.2 s	1/1				59.0 B

Hình 13. Spark UI, trang Stages: các stage đọc JDBC song song và stage ghi Parquet đều có 4 task.



Hình 14. Đầu ra Parquet phân vùng theo tháng trên HDFS và so sánh dung lượng với đầu ra văn bản của Sqoop.

2.5 Spark nạp gia tăng bằng watermark

Bước `incremental` chốt cận trên bằng `now()` của cơ sở dữ liệu trước khi đọc, đọc cửa sổ `(wm_old, wm_new]` theo `updated_at`, nối vào change log `orders_changes`, dựng snapshot `orders_current` bằng `row_number()` theo `order_id` (sắp `updated_at DESC, _ingested_at DESC`), và chỉ ghi watermark sau cùng:

```
wm_new = jdbc(spark, query="SELECT to_char(now(), 'YYYY-MM-DD
 ↪  HH24:MI:SS.US') AS ts").first().ts
changed_sql = (f"(SELECT * FROM orders WHERE updated_at > timestamp
 ↪  '{wm_old}'"
               f" AND updated_at <= timestamp '{wm_new}') AS
                ↪  changed_orders")
# ... ghi changes (kèm cột _ingested_at) vào orders_changes, rồi dựng
 ↪  snapshot:
log = spark.read.parquet(f"{BASE}/orders_changes")
w = Window.partitionBy("order_id") \
    .orderBy(F.col("updated_at").desc(), F.col("_ingested_at").desc())
current = log.withColumn("_rn", F.row_number().over(w)).filter("_rn =
 ↪  1").drop("_rn")
current.write.mode("overwrite").parquet(f"{BASE}/orders_current")
spark.createDataFrame([(wm_new,)], "watermark string").coalesce(1) \
    .write.mode("overwrite").json(f"{BASE}/_state/orders_watermark")
```

Kết quả ba lần chạy ở Bảng 9 và Hình 15.

Bảng 9. Kết quả ba lần chạy Spark nạp gia tăng (thời điểm ngày 2026-10-30).

Lần chạy	Cửa sổ watermark	Dòng đọc được	Change log	Snapshot
1 (chưa có watermark)	`(1970-01-01 00:00:00, 2026-10-30 10:51:08.121235]`	5,003	5,003	5,003
`simulate_changes` lần 2	tạo 5004–5006, cập nhật đơn 337 và 355			
2	`(10:51:08.121235, 10:51:26.913767]`	**5**	5,008	5,006
3 (không có thay đổi)	`(10:51:26.913767, 10:51:44.317579]`	0	5,008	5,006

Change log giữ mọi phiên bản ($5003 + 5 = 5008$), còn snapshot chỉ giữ phiên bản mới nhất của mỗi đơn ($5003 + 3 = 5006$), nên hai đơn được cập nhật không bị nhân đôi. Lần thứ ba đọc 0 dòng: watermark được lưu đúng và chạy lại không làm sai kết quả. Cận trên của lần trước trùng

khít với cận dưới của lần sau.

L04 — Spark incremental (watermark updated_at) × 3 lần

```
### run 1 (no watermark yet)
$ spark-submit spark_jdbc_ingest.py --step incremental
watermark: (1970-01-01 00:00:00, 2026-10-30 10:51:08.121235]
changed rows since last run: 5003
+---------+--------------------------------+------------+-------------------+
|order_seq|order_id                        |order_status|updated_at         |
+---------+--------------------------------+------------+-------------------+
|13       |8fcda0cbaf5fbdf74b44ec00ad898b16|invoiced    |2017-01-04 08:28:34|
|9        |3e72f2038b375f7cffb84634e058a90d|delivered   |2017-01-08 03:33:57|
|10       |04e531031c4460209ab391b1ecd08b73|delivered   |2017-01-09 14:43:54|
|14       |b9215ed78670f5fcdd902dd20d25ddfa|delivered   |2017-01-10 08:30:04|
|1        |7fde1c0d7fe0422f9f39c260b2a00fe0|delivered   |2017-01-11 06:01:41|
|7        |4dc8028f24056a2745258e340b6f0acb|delivered   |2017-01-11 17:59:16|
|16       |a8b568029838e24b62999f248b689a46|delivered   |2017-01-12 10:25:07|
|5        |45358698bf0058ceccd0ba051b389c3e|delivered   |2017-01-12 18:46:29|
|11       |78565a07e33b7cee0300e9a0dd9b165d|delivered   |2017-01-12 20:29:04|
|4        |94210b260a31271221a4b2385b509571|delivered   |2017-01-13 14:17:16|
+---------+--------------------------------+------------+-------------------+
only showing top 10 rows
change-log rows: 5003 | current snapshot rows: 5003
saved watermark 2026-10-30 10:51:08.121235

### $ psql -f sql/simulate_changes.sql   (#2)
INSERT 0 3
UPDATE 2
 order_seq |             order_id             | order_status |         updated_at
-----------+----------------------------------+--------------+----------------------------
      5004 | 7fcdf5479024a8742073bb0ef6935142 | created      | 2026-10-30 10:51:17.171709
      5005 | 074912012cd67af7286f9d65fda97044 | created      | 2026-10-30 10:51:17.171709
      5006 | 87aa76620ff952a3c59e109b75a627fb | created      | 2026-10-30 10:51:17.171709
       337 | 9914cbc8175f775efa350b76cb42c257 | delivered    | 2026-10-30 10:51:17.175487
       355 | d95630b6b34b9e5fc509a5d750b0fa66 | delivered    | 2026-10-30 10:51:17.175487
(5 rows)

### run 2 (after simulate_changes)
$ spark-submit spark_jdbc_ingest.py --step incremental
watermark: (2026-10-30 10:51:08.121235, 2026-10-30 10:51:26.913767]
changed rows since last run: 5
+---------+--------------------------------+------------+--------------------------+
|order_seq|order_id                        |order_status|updated_at                |
+---------+--------------------------------+------------+--------------------------+
|5004     |7fcdf5479024a8742073bb0ef6935142|created     |2026-10-30 10:51:17.171709|
|5005     |074912012cd67af7286f9d65fda97044|created     |2026-10-30 10:51:17.171709|
|5006     |87aa76620ff952a3c59e109b75a627fb|created     |2026-10-30 10:51:17.171709|
|337      |9914cbc8175f775efa350b76cb42c257|delivered   |2026-10-30 10:51:17.175487|
|355      |d95630b6b34b9e5fc509a5d750b0fa66|delivered   |2026-10-30 10:51:17.175487|
+---------+--------------------------------+------------+--------------------------+
change-log rows: 5008 | current snapshot rows: 5006
saved watermark 2026-10-30 10:51:26.913767

### run 3 (no changes)
$ spark-submit spark_jdbc_ingest.py --step incremental
watermark: (2026-10-30 10:51:26.913767, 2026-10-30 10:51:44.317579]
changed rows since last run: 0
+---------+--------+------------+----------+
|order_seq|order_id|order_status|updated_at|
+---------+--------+------------+----------+
+---------+--------+------------+----------+
change-log rows: 5008 | current snapshot rows: 5006
saved watermark 2026-10-30 10:51:44.317579
```

Hình 15. Ba lần chạy Spark `--step incremental`.

2.6 So sánh Sqoop và Spark JDBC

Bảng so sánh của Phần 3 trong đề, kèm thời gian đo được, nằm ở Bảng 10.

Bảng 10. So sánh Sqoop và Spark JDBC khi nạp bảng `orders`.

Tiêu chí	Sqoop 1.4.7 (MapReduce)	Spark JDBC
Tham số song song	`-m 4` + `--split-by`	`numPartitions=4` + `partitionColumn`
Cách tính khoảng	Sqoop tự chạy `SELECT MIN, MAX` (BoundingValsQuery)	người dùng truyền `lowerBound`/`upperBound` (chỉ là stride)
Nạp gia tăng	Có sẵn: `--incremental append/lastmodified`, saved job lưu last value	Tự viết: truy vấn watermark + file trạng thái
Xử lý UPDATE	`--merge-key` (thêm một MR job)	`row_number()` khử trùng lặp, hoặc MERGE vào table format
Định dạng đầu ra	Văn bản CSV, 1.0 MB	Parquet Snappy phân vùng theo tháng, 680.4 KB
Biến đổi khi nạp	Không (chỉ SQL qua `--query`)	Đầy đủ DataFrame API (`withColumn`, `partitionBy`, …)
Thời gian nạp `orders`	25.2 s MR job (28.5 s cả lệnh)	3.10 s ghi Parquet (21.2 s cả `spark-submit` 6 bảng, gồm khởi động JVM)
Trạng thái dự án	Apache Attic (ngừng từ 2021)	Đang phát triển

Sqoop chậm chủ yếu vì mỗi lệnh phải sinh mã, biên dịch lớp record và dựng một MapReduce job riêng. Spark dùng một SparkSession cho cả 6 bảng và ghi thẳng ra định dạng cột có nén, nên đầu ra nhỏ hơn khoảng một phần ba. Tuy vậy mọi thứ chạy trên một máy với khoảng 1 MB dữ liệu, thời gian đo phản ánh chi phí khởi động nhiều hơn thông lượng, nên chênh lệch thời gian chỉ nên hiểu theo nghĩa định tính.

3 Trả lời câu hỏi checkpoint

3.1 Sqoop chạy câu SQL nào trước khi import, và từ đó ra 4 split bằng cách nào?

Sqoop chạy bounding query `SELECT MIN("order_seq"), MAX("order_seq") FROM "orders"` và nhận (1, 5000) (Hình 3). Bộ chia số nguyên cắt đoạn này thành 4 khoảng bằng

nhau về giá trị, rộng khoảng $(5000 - 1)/4 \approx 1250$; mỗi map chạy `WHERE order_seq >= lo AND order_seq < hi`, khoảng cuối dùng <= để lấy cả MAX (Bảng 3). Vì chia theo giá trị chứ không theo số dòng, các split chỉ đều khi cột chia phân bố đều.

3.2 Vì sao các file `customers` không đều, và nếu dùng `--split-by customer_id` thì sao?

Sqoop chia đều theo giá trị, nhưng zip code của khách hàng Olist dồn về São Paulo, nên khoảng đầu [1014, 24536) nhận gần nửa bảng còn khoảng [48057, 71578) rất ít (Bảng 4, Bảng 5). Đây là data skew: task nặng nhất quyết định thời gian của cả job. Còn `customer_id` là chuỗi hash kiểu text; mặc định Sqoop từ chối chia theo cột text và báo lỗi, trừ khi bật `-Dorg.apache.sqoop.splitter.allow_text_splitter=true`. Kể cả khi bật, việc chia theo thứ tự chuỗi phụ thuộc collation nên split vẫn có thể lệch hoặc rỗng. Cột chia tốt nhất là cột số, phân bố đều và có chỉ mục, như `order_seq`.

3.3 Với `lowerBound=1, upperBound=5003, numPartitions=4`, Spark có đọc dòng `order_seq` = 9,999 không?

Có đọc. `lowerBound` và `upperBound` chỉ dùng để tính bước chia, không phải bộ lọc: partition cuối là `order_seq >= 3752` không có cận trên (Hình 10), nên dòng 9,999 rơi vào partition cuối; giá trị nhỏ hơn `lowerBound` và NULL rơi vào partition đầu. Cận sai không làm mất dữ liệu, nhưng partition hai đầu sẽ phình to và gây skew. Vì vậy script lấy MIN/MAX thật trước khi đọc, nhờ đó 4 partition gần đều (Bảng 7).

3.4 Vì sao phải chốt cận trên trước khi đọc, và chỉ lưu watermark sau khi ghi xong?

Script lấy `wm_new = now()` trước khi đọc, rồi truy vấn `updated_at > wm_old AND updated_at <= wm_new` (Mục 2.5). Nếu lấy `now()` sau khi đọc, dòng được cập nhật trong lúc đang đọc sẽ có `updated_at` nhỏ hơn watermark mới mà không nằm trong dữ liệu vừa đọc, và lần sau bị bỏ qua luôn. Cận trên cố định còn bảo đảm các partition JDBC song song cùng nhìn một cửa sổ, và các cửa sổ liên tiếp liền nhau mà không chồng lên nhau, đúng như cận của ba lần chạy ở Bảng 9.

Về thứ tự lưu, nếu job chết giữa chừng thì watermark cũ vẫn còn và lần chạy lại đọc lại cả cửa sổ. Dữ liệu có thể trùng nhưng không mất (*at-least-once*), và phần trùng được loại bằng `row_number()` khi dựng snapshot. Ngược lại, nếu lưu watermark trước rồi bước ghi thất bại, cửa sổ đó sẽ không bao giờ được đọc lại.

3.5 Chế độ append hay lastmodified bắt được UPDATE, và chế độ nào bắt được DELETE?

append chỉ dựa vào order_seq > last_value nên chỉ bắt được INSERT: saved job chỉ lấy các đơn mới (Bảng 6). lastmodified dựa vào updated_at nên bắt được cả INSERT và UPDATE (thêm đơn 139 và 278); khi đó cần --merge-key để gộp bản cũ với bản mới. Không chế độ nào bắt được DELETE, vì dòng đã xóa không còn trong bảng để truy vấn.

3.6 Watermark trên updated_at không giải quyết được gì, và CDC khác ở đâu?

Thứ nhất là DELETE: dòng bị xóa không bao giờ xuất hiện trong truy vấn, nên bản sao trên HDFS giữ lại dữ liệu đã không còn ở nguồn, trừ khi ứng dụng dùng soft delete hoặc so sánh cả snapshot. Thứ hai là các lần UPDATE trung gian: nếu một đơn đổi từ shipped sang delivered rồi canceled trong cùng một chu kỳ, chỉ trạng thái cuối được thấy. Ngoài ra updated_at có thể không đáng tin: ứng dụng quên cập nhật cột, đồng hồ máy chủ lệch, hoặc một transaction dài commit sau khi watermark đã vượt qua timestamp của nó và dòng đó bị sót. Cách này cũng tạo tải quét lên nguồn (cần chỉ mục trên updated_at), và độ trễ bằng chu kỳ lập lịch.

CDC (Debezium, sẽ làm ở L08) không truy vấn bảng mà đọc write-ahead log (WAL) của PostgreSQL qua logical decoding [3], [4]. Mọi INSERT, UPDATE và DELETE thành sự kiện theo thứ tự commit, kèm ảnh trước và sau, không phụ thuộc updated_at, gần như không tạo tải truy vấn và trễ chỉ vài giây.

3.7 Điều gì xảy ra với cơ sở dữ liệu nguồn khi đặt -m 50 hoặc numPartitions=50?

Nguồn nhận 50 kết nối đồng thời, mỗi kết nối một range scan. Con số này dễ chạm max_connections (PostgreSQL mặc định 100) khiến ứng dụng production không lấy được kết nối, và các truy vấn tranh CPU, I/O, buffer cache với tải OLTP; nếu cơ sở dữ liệu đã là nút thắt thì tốc độ gần như không tăng. Với bảng 5,000 dòng, mỗi task chỉ còn khoảng 100 dòng, chi phí khởi tạo lớn hơn lợi ích. Thực tế nên giữ khoảng 4–8 kết nối, thống nhất với DBA, đọc từ read replica hoặc ngoài giờ cao điểm.

4 Kết luận

Bài đã hoàn thành đủ phần Sqoop (nạp toàn phần, cột chia lệch, append, saved job, lastmodified kèm merge, export), phần Spark JDBC (đọc song song, Parquet phân vùng theo tháng, nạp gia tăng bằng watermark) và bảng so sánh. Nhật ký SQL cho thấy cả hai công cụ chia bảng theo khoảng giá trị, nên order_seq cho các task đều còn zip code và timestamp thì

lệch. `append` chỉ bắt INSERT, `lastmodified` và watermark bắt thêm UPDATE, còn DELETE và phiên bản trung gian cần CDC.

Tài liệu tham khảo

[1] The Apache Software Foundation, “Sqoop User Guide (v1.4.7),” 2017. https://sqoop.apache.org/docs/1.4

[2] The Apache Software Foundation, “JDBC To Other Databases,” Spark SQL Guide. https://spark.apache.org/docs/latest/sql-data-sources-jdbc.html

[3] The PostgreSQL Global Development Group, “Logical Decoding,” PostgreSQL Documentation. https://www.postgresql.org/docs/current/logicaldecoding.html

[4] Debezium Community, “Debezium Connector for PostgreSQL,” Debezium Documentation. https://debezium.io/documentation/reference/stable/connectors/postgresql.html

Phụ lục: Tệp nộp kèm

Đoạn mã tự viết chính đã trích trong Mục 2.5; tệp đầy đủ ở Bảng 11.

Bảng 11. Danh mục tệp nộp kèm báo cáo L04.

Tệp	Nội dung
`sqoop_part1.sh`, `spark_jdbc_ingest.py`	Lệnh Sqoop phần 1; script Spark JDBC đã hoàn thành
`01_*.txt` đến `09_*.txt`, `img/`	Log terminal, nhật ký SQL của PostgreSQL, ảnh chụp terminal và UI